

Does encrypting our AI keys slow the app down?

Benchmark report on how Homenet gets and unlocks its Groq API keys, measured over 1,000 operations against the real code and database.

Short answer: No. Keys are unlocked (decrypted) **once**, when the server starts. After that, no request ever decrypts a key. Unlocking a key takes about **38 millionths of a second**, far too small for anyone to notice.

0

keys decrypted per
user request

0.038 ms

to decrypt one key
(done once at startup)

~52 ms

per AI call, spent on one
database trip (network)

In plain words

Think of the API keys as house keys kept in a locked safe (the database). The master key opens the safe. When the server starts its day, it opens the safe once, takes out all the keys and puts them in its pocket (memory). Whenever a user needs the AI, the server just takes the next key from its pocket. It does not walk back to the safe each time.

The only thing the server does check on every AI call is a shared notebook in the database: whose turn is it, and is any key resting? That quick check is what costs about 52 ms, and almost all of that time is the message travelling to the database and back, not the work itself.

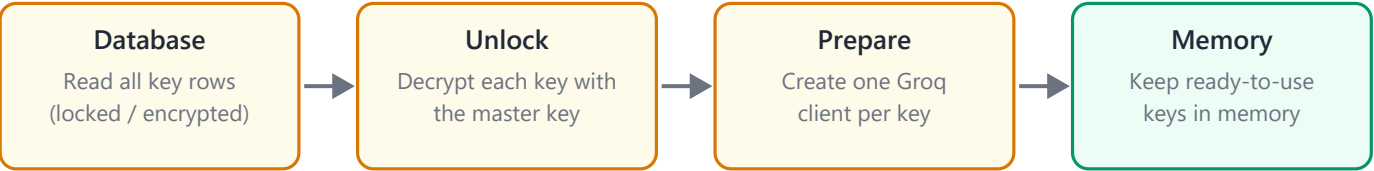
Key takeaways

#	Takeaway
1	No request fetches or decrypts a key. All keys are decrypted once per server start and kept in memory.
2	Each AI call costs one database round trip (~52 ms from a local machine). That is network distance; the actual computer work is under 1 ms.
3	Decryption is negligible: 38 µs per key. 1,000 decryptions take 0.04 seconds in total.
4	Keeping keys in memory saves time: fetching and decrypting on every request would add a second ~52 ms trip to every AI call.
5	Under heavy load (20 requests at once) the system handles ~230 key look-ups per second. The occasional slow one waits for a free database connection, not for encryption.

How it works

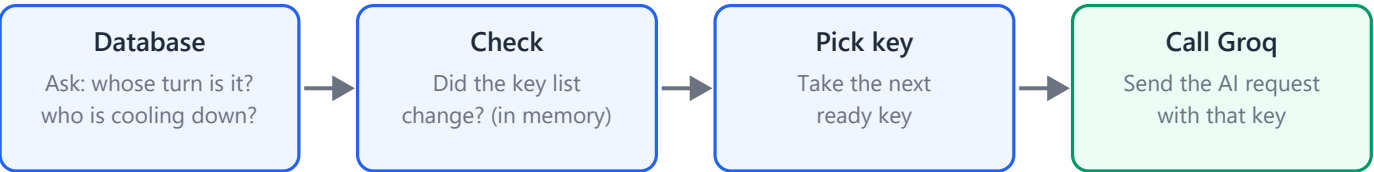
Two separate moments: once at startup, and on every AI request.

ONCE, when the server starts



Cost: ~53 ms in total (one database read). Decrypting the 3 keys takes about 0.1 ms of that.

EVERY time a user's request needs the AI



Cost: ~52 ms (one database read, mostly network distance). No key is fetched and nothing is decrypted.

What is stored where

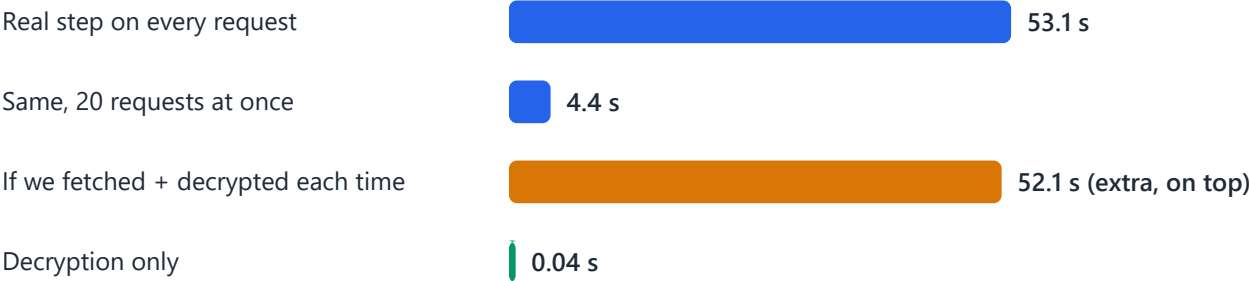
Item	Where it lives	Form
Groq API keys	Database table llm_api_keys	Encrypted (AES-256-GCM). Unreadable without the master key.
Master key	Server setting LLM_MASTER_ENCRYPTION_KEY	Never stored in the database.
Unlocked keys	Server memory, after startup	Ready to use. Never sent to users, never written anywhere.
Users' browsers / apps	None	Never receive a key.

When are keys unlocked again? Only when the server restarts, or when an admin changes the list of keys (for example by re-seeding them). The server notices the change on the next AI call and reloads automatically.

The results

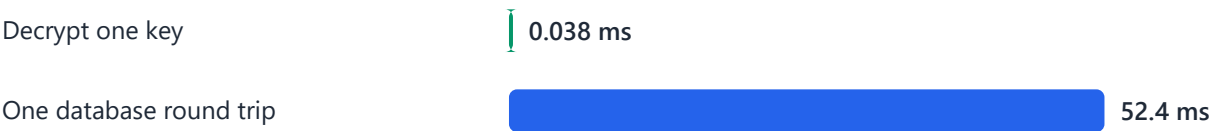
1,000 operations per test, run on 1 October 2026 against the real app code and the Neon database (3 keys). No AI requests were sent, so no AI quota was used.

Total time for 1,000 operations



Total time for 1000 operations (shorter is better)

One operation, side by side



Typical time for one operation. Decryption is about 1,400 times faster than one database trip.

Full numbers

Test	What it does	Typical (median)	Slow 1-in-10 (p90)	Slowest 1-in-100 (p99)	1,000 ops total
A	Real step on every AI call: database check + pick key	52.4 ms	53.3 ms	77.5 ms	53.1 s
A2	Same as A, 20 at once	79.4 ms	92.7 ms	502 ms	4.4 s
B	What we avoid: fetch key + decrypt on every call	51.6 ms	52.6 ms	77.8 ms	52.1 s
C	Decrypt one key only	0.038 ms	0.040 ms	0.073 ms	0.04 s
D	Full startup load: all keys fetched + decrypted (20 runs)	52.9 ms	53.6 ms	53.8 ms	1.05 s / 20

Computer work (CPU) per operation: A 0.84 ms, A2 1.2 ms, B 1.6 ms, C 0.032 ms, D 10 ms. Memory growth during each test was 0.7 to 3.7 MB of short-lived data that is cleaned up automatically.

What we learned

#	Finding
1	Network is the cost, not encryption. A simple 'hello' query to the database takes the same ~50-120 ms from this machine. The real work on our side is under 1 ms per call.
2	Encryption is not a performance factor. One decryption is about 0.07% of a single database trip. Unlocking all 3 keys takes ~0.1 ms, and even 1,000 keys would take ~48 ms and under 1 MB of memory, once at startup.
3	Keeping keys in memory pays off. Without it, every AI call would need two database trips instead of one: about 104 ms instead of 52 ms. A search (up to 2 AI calls) would lose about 0.1 seconds.
4	Heavy load is limited by database connections. With 20 requests at once, throughput is ~230 per second. The rare slow one (~0.5 s) is waiting for a free database connection. Allowing more connections would help; encryption plays no part.
5	Reloading keys is cheap. A full reload takes ~53 ms (one database read), so updating keys has no meaningful cost for a running server.

Conclusion

The encryption design adds **no measurable delay** to user requests. Keys are decrypted once at startup in microseconds. The only per-request cost is one shared database check, which keeps key rotation and rate-limit pauses consistent across all servers. Its speed depends on how far the server is from the database (~52 ms locally; expected to be a few ms on Vercel in the same region as Neon, not yet measured).

No changes are recommended. If requests need to be faster, run the servers in the same region as the database.

Words used in this report

Term	Meaning
Decrypt	Unlock an encrypted key using the master key, so it can be used.
Round trip	A message going from the server to the database and the answer coming back.
ms / μ s	Millisecond = 1/1,000 of a second. Microsecond (μ s) = 1/1,000,000 of a second.
Median	The typical time: half of the operations were faster, half slower.
p90 / p99	90% / 99% of operations were at least this fast; shows the slow ones.
Concurrency 20	20 operations running at the same time, like 20 users at once.

Side effects of running this test: the shared rotation counter moved forward by about 1,020, the same as 1,020 real AI calls; rotation simply continues from there. Nothing else was changed, no AI requests were made, and decrypted keys were never displayed. The technical version of this report, including the script used, is [docs/arman-docs/llm-key-benchmark-report.md](#).